

How to Prepare for D2 Progress Check

Milestone D2 is an early progress check. At this stage, you should have a **near-complete implementation of at least one of User Service or Supplier Service, and made significant progress on the other.** Your team should demonstrate clear design decisions, evidence of implementation, and an understanding of how these services will support the later Order and Credit workflows.

Time limit: 20-30 min

General Tips

- Come prepared with a brief live demo, supporting diagrams (e.g., component-level architecture, sequence diagrams, workflows, etc.) and a concise explanation of your decisions.
- Prepare for "why" questions — your mentor will probe the reasoning behind your decisions

Part 1: User Service

Your task is to develop the User Service with user roles and role-based access control (RBAC).

Points 1-6 are the expectations for a **near-complete** user service.

Points 1-4 are the expectations for **significant progress** on the user service.

1. Role Design

- What roles have you defined for FoC?
- Explain *why* these roles are appropriate for the platform and what problem each role solves.
- In your presentation, clearly document each role's capabilities, including the actions it can perform and the parts of the system to which its permissions apply.
 - Update this artifact as your design evolves, so that you can use it in the final project presentation later.

2. Database Choice and Schema Design

- Which database technology have you selected? Justify your choice based on FoC's specific needs, including the nature of the user data, expected query patterns, and scalability requirements.

- Present a concrete database schema. Clearly describe the tables, collections, or documents; the fields they contain; and the relationships among them.
 - Explain how authentication credentials are stored and managed securely.
3. Authentication and Authorization
- How are users authenticated by the User Service? Explain the approach used (e.g., token-based authentication or a third-party identity service).
 - Implement role-based access control (RBAC) to enforce the permissions defined for each role. Explain how authorization is enforced after authentication (e.g., are permissions enforced through token claims, middleware or authorization checks)? Be ready to justify why the approach suits your design of FoC.
 - Demonstrate the running User Service, including how authentication and role-based access control are enforced.
4. Integration with the Supplier Service
- Be prepared to demonstrate how the User Service is used by the Supplier Service to enforce access restrictions. This demonstration should also provide evidence that role-based access control is working as intended.
5. User Profile Management
- Explain how profile updates are validated and protected.
 - How are users prevented from modifying protected information, such as their role, account status, or user ID?
6. Role Lifecycle & Administration
- How is the first administrator created? Explain how this process is controlled and secured.
 - How does a user get *promoted* to a higher-privilege role? How can a user be promoted to a higher-privilege role? Describe the complete workflow, ensuring that it does not require manual developer intervention for every promotion.
 - Consider edge cases in role management. For example:
 - What happens when an administrator attempts to revoke their own privileges?
 - What happens when the only administrator in the system attempts to delete or demote their account?

Part 2: Supplier Service

Your task is to implement the Supplier Service backend, together with a responsive UI. The UI should consume the Supplier Service's APIs to perform CRUD operations.

Points 1-5 are the expectations for a **near-complete** supplier service.

Points 1-4 are the expectations for **significant progress** on the supplier service.

1. Database Choice and Schema Design
 - Which database technology have you selected?
 - Justify your choice with reasoning specific to FoC's design. Consider the nature of the data—for example, whether it is structured or contains flexible content—as well as the expected query patterns and scalability requirements.
 - Present a concrete schema. Specify the tables, collections or documents; the fields they contain; and the relationships among them.
 - Identify the metadata required for each supplier, such as its name, type and location, and explain how this metadata is stored and queried.
2. Query Patterns & API Design
 - Identify the key ways in which the Supplier Service will be queried, such as retrieving a supplier by ID or finding suppliers by location.
 - Define the API endpoints that expose these operations. Demonstrate the key query patterns using working API calls.
 - Explain how Supplier Service API endpoints use identity and role information from the User Service to enforce access control and respond to denied requests.
3. CRUD operations in the Backend
 - Demonstrate a running Supplier Service connected to a database, with working APIs that support CRUD operations on supplier data.
 - The Supplier Service is an independent backend service. Its functionality must be exposed through its APIs and should not depend on the UI.
 - Demonstrate that the Supplier Service can be used and tested through its APIs without the UI being present or running.
4. End-to-End Integration
 - Demonstrate the complete flow from an authenticated user, through the Supplier Service API, to the database.
 - Show that users with different roles receive the appropriate API access.
 - Focus the demonstration on the complete supplier-management experience for all relevant users, including administrators where applicable.
5. Responsive Supplier Management UI

- Demonstrate the complete supplier management UI.
- Show the UI adapting appropriately across desktop- and mobile-sized viewports.
- Include workflows such as:
 - Creating, editing, and deleting suppliers
 - Viewing supplier records
 - Searching for suppliers
 - Filtering and sorting results
 - Paginating through supplier data
 - Viewing supplier details
- Demonstrate integration with the Supplier Service by retrieving and modifying supplier data through its APIs. The UI must use live data from the service and must not rely on mock or hard-coded supplier data.
 - Note: The demonstration should not be limited to administrator-only screens.